Prediction, Compression, and Intelligence: Three Names for the Same Operation

Section 3 — Philosophy of Machine Minds
Executive Education Lecture Notes

Preface: how to read these notes

These notes are written on two tracks at once, for two readers travelling together. The *student* — typically a bachelor-STEM reader who wants the theorems, proofs, and definitions — will find the main text body rigorous and self-contained. The *manager* — a decision-maker who wants the conceptual story without the symbols — will find every formal object wrapped in a shaded **Intuition** box written in plain English, with analogies, small numerical examples, and a "why should a decision-maker care" coda. Each Intuition box cross-references the nearest definition or theorem via Section 1. . . so a curious manager can always drop down into the math, and a rigorous student can always drop up into the metaphor. Read strictly down the page and you get both tracks interleaved; read only the Intuition boxes and the section headings and you will still leave with the thesis: *prediction, compression, and intelligence are, under precise definitions, the same operation, and that fact is the implicit business plan of every major foundation-model lab.*

Contents

1 Why this equivalence matters **2**

2 Prelude: Shannon and the cost of surprise **3**

3 Prediction ⇔ Compression (Shannon's half of the bridge) **4**

4 Kolmogorov complexity **6**

5 Solomonoff induction **8**

6 AIXI: intelligence as optimal compression-driven prediction **10**

7 What this buys us for LLMs **11**

8 Caveats and what the equivalence does not say **13**

Further reading **15**

A Worked cross-entropy calculation **16**

1 Why this equivalence matters

Intuition

The claim we unpack in these notes, in one sentence: *being good at predicting what comes next, being good at compressing a file, and being good at thinking are — up to translation — the same skill.* That sounds like a philosopher's trick. It is not. It is a chain of three mathematical results, each modest on its own, that lock together like gears. Shannon showed prediction and compression are literally the same job in different units (Theorem 3.2). Kolmogorov showed that "the shortest description of a thing" is a deep, coordinate-free notion of what the thing *is* (Theorems 4.1 and 4.2). Solomonoff and Hutter showed that an agent who bets according to "shorter explanations are more likely" is, in a precise sense, the smartest possible learner (Theorems 5.2 and 6.2).

Why should a decision-maker care? Because your vendors are selling you products whose entire training objective is one single number: how surprised the model was by the next word of the internet. If you believe the chain above, minimising that one number is not a narrow engineering goal — it is a proxy for general intelligence. That is either the most important business fact of the decade or a confusion between two different things sharing a name. You cannot tell which without understanding the equivalence. Think of it as three currencies — **bits saved** (how tightly a model compresses your data), **dollars earned** (what that compression is worth when deployed), and **capability on held-out tasks** (how well the model handles problems it has not seen) — and three exchange rates between them. Three currencies, three exchange rates, one underlying quantity. Once you see the rates, a lot of hype becomes legible, and a lot of legitimate excitement stops sounding like magic.

One frame to carry throughout. A perfect next-token predictor would be an ideal learner; real next-token predictors are not perfect, and the gap is exactly where this document's caveats live (Section 8). We state the thesis before we qualify it, but the qualification is not optional.

The central technical claim of this note is that three operations that are often treated as distinct — predicting the next symbol in a sequence, compressing that sequence into a shorter description, and behaving intelligently in its presence — are, under a precise set of definitions, the *same operation* viewed from three angles. Shannon's source coding theorem [Shannon, 1948] identifies prediction with compression; Kolmogorov complexity [Kolmogorov, 1965] makes the notion of "shortest description" independent of the choice of language; Solomonoff's universal prior [Solomonoff, 1964] promotes the Kolmogorov minimiser to a Bayesian predictor that converges to any computable source; and Hutter's AIXI [Hutter, 2005] upgrades that predictor into a reward-seeking agent whose optimality can be stated as a theorem. The chain is not an analogy. Every step in this note is a definition, a lemma, or a theorem. The pedagogical payoff is that one can then ask sharp questions about large language models — whether they count as compressors, whether better compression entails more general capability, and where the equivalence breaks — without smuggling in metaphor.

2 Prelude: Shannon and the cost of surprise

Intuition

Information is the cost of being surprised, and you are surprised exactly as much as an event was unlikely. In Los Angeles, a forecaster who says "sunny" every day will be right most of the time and needs almost no words. In Zurich, she needs rain, snow, Föhn, fog, and sun. The *average* number of yes/no questions she needs to convey a day's weather is the *entropy* (Theorem 2.1), denoted $H(X)$. Flip a fair coin: one yes/no question suffices, so $H = 1$ bit. Flip a coin biased 7/8 heads: most of the time you guess heads and are done; on average you need only about half a bit per flip. A 1-in-1024 event carries ten bits of "wow"; a 50/50 event carries one bit of "huh."

Shannon's source-coding theorem (Theorem 2.4), in plain English: you cannot, on average, describe a random source using fewer bits than its entropy; and you can get within one bit of entropy if you are clever. Entropy is the floor; clever coding reaches it.

Why a decision-maker should care: every storage bill, every bandwidth bill, every "our model is more efficient" vendor claim bottoms out against this floor. If somebody tells you they compress English text to 0.3 bits per character, they are either wrong, lying, or have discovered English has lower entropy than anyone thought — which would itself be news. Entropy also sets the floor on forecasting: no analyst, human or machine, can predict a genuinely high-entropy process (next week's lottery numbers) better than chance, no matter how much compute you throw at it. Knowing where the entropy floor sits for *your* data is the difference between a reasonable AI budget and a fantasy one.

Fix a finite *alphabet* $\mathcal{A} = \{a_1, \ldots, a_K\}$ and let X be a discrete random variable on $\mathcal{A}$ with probability mass function $p(a) = \Pr[X = a]$. All logarithms are base 2 unless stated; values are measured in *bits*.

Definition 2.1 (Self-information and Shannon entropy)**.** The **self-information** (or *surprisal*) of the outcome $x \in \mathcal{A}$ under p is

$$I_p(x) \;=\; -\log_2 p(x), \tag{1}$$

with the convention $0 \cdot \log_2 0 = 0$. The **Shannon entropy** of X is the expected self-information,

$$H(X) \;=\; \mathbb{E}\Big[I_p(X)\Big] \;=\; -\sum_{k=1}^{K} p(a_k)\,\log_2 p(a_k). \tag{2}$$

A **code** for $\mathcal{A}$ is an injective map $C : \mathcal{A} \to \{0,1\}^*$ assigning a binary string $C(a)$ to each symbol. We write $\ell(a) = |C(a)|$ for the code length. C is a **prefix code** (or *instantaneous code*) if no codeword $C(a_i)$ is a proper prefix of any other codeword $C(a_j)$.

Lemma 2.2 (Kraft's inequality)**.** *For any prefix code C on $\mathcal{A}$ with lengths $\ell(a_k)$,*

$$\sum_{k=1}^{K} 2^{-\ell(a_k)} \;\le\; 1. \tag{3}$$

Conversely, for any length assignment $\ell_1, \ldots, \ell_K \in \mathbb{N}$ satisfying (3) *there exists a prefix code with those lengths.*

Proof sketch. Identify each codeword with a leaf in the infinite binary tree. A prefix condition forbids any codeword from lying on the path to another, so distinct codewords correspond to disjoint subtrees. The subtree rooted at a node of depth ℓ contains a $2^{-\ell}$ fraction of the infinite leaves, and disjointness gives the bound. The converse is an explicit construction: sort ℓ_k in increasing order and assign the lexicographically smallest available length-ℓ_k string at each step. □

Lemma 2.3 (Gibbs' inequality)**.** *For any two probability mass functions p, q on $\mathcal{A}$,*

$$-\sum_k p(a_k)\log_2 q(a_k) \;\geq\; -\sum_k p(a_k)\log_2 p(a_k), \tag{4}$$

with equality if and only if $p = q$.

Proof. Using $\ln x \leq x - 1$ (with equality iff $x = 1$) and $\log_2 x = \ln x/\ln 2$,

$$\sum_k p(a_k)\log_2\frac{q(a_k)}{p(a_k)} \;\leq\; \frac{1}{\ln 2}\sum_k p(a_k)\left(\frac{q(a_k)}{p(a_k)} - 1\right) \;=\; \frac{1}{\ln 2}\left(\sum_k q(a_k) - 1\right) \;\leq\; 0,$$

because $\sum_k q(a_k) \leq 1$. Rearranging gives (4); the equality conditions of $\ln x \leq x - 1$ and $\sum q = 1$ together force $p = q$. □

Theorem 2.4 (Shannon source coding inequality)**.** *Let C be any uniquely decodable (in particular, any prefix) code for $\mathcal{A}$ and let $L = \mathbb{E}[\ell(X)] = \sum_k p(a_k)\ell(a_k)$ be its expected code length. Then*

$$L \;\geq\; H(X). \tag{5}$$

Moreover, there exists a prefix code (e.g. the Shannon–Fano code with $\ell(a_k) = \lceil -\log_2 p(a_k)\rceil$) such that

$$L \;<\; H(X) + 1. \tag{6}$$

Proof sketch. Let $q(a_k) = 2^{-\ell(a_k)}/Z$, where $Z = \sum_k 2^{-\ell(a_k)} \leq 1$ by Theorem 2.2, so $\tilde{q} = q$ is a bona fide probability vector after normalisation. By Theorem 2.3, the cross-entropy $-\sum_k p(a_k)\log_2\tilde{q}(a_k) \geq H(X)$. Because $-\log_2\tilde{q}(a_k) = \ell(a_k) + \log_2 Z \leq \ell(a_k)$ (since $Z \leq 1$ makes $\log_2 Z \leq 0$), we get $L \geq -\sum_k p(a_k)\log_2\tilde{q}(a_k) \geq H(X)$. For (6), the lengths $\lceil -\log_2 p(a_k)\rceil$ satisfy Kraft (3), and each ceiling adds at most 1 to the expected length. See Cover and Thomas [2006, Ch. 5] for the full treatment. □

Example 2.5 (Entropy of a 4-symbol alphabet)**.** Let $\mathcal{A} = \{a, b, c, d\}$ with $p(a) = \frac{1}{2}$, $p(b) = \frac{1}{4}$, $p(c) = p(d) = \frac{1}{8}$. Then

$$\begin{aligned} H(X) &= -\tfrac{1}{2}\log_2\tfrac{1}{2} - \tfrac{1}{4}\log_2\tfrac{1}{4} - 2\cdot\tfrac{1}{8}\log_2\tfrac{1}{8} \\ &= \tfrac{1}{2}\cdot 1 + \tfrac{1}{4}\cdot 2 + \tfrac{1}{4}\cdot 3 \;=\; \tfrac{7}{4}\text{ bits.} \end{aligned}$$

A prefix code achieving $L = H(X) = 1.75$ exactly is $C(a) = 0,\ C(b) = 10,\ C(c) = 110,\ C(d) = 111$; its lengths $(1, 2, 3, 3)$ satisfy Kraft with equality and match $-\log_2 p(a_k)$ symbol by symbol.

3 Prediction ⇔ Compression (Shannon's half of the bridge)

Intuition

A predictor that assigns probabilities to the next word is, mechanically, a compressor; its error rate and its compression rate are the same number in different clothes (see Theorem 3.2 and Theorem 3.3).

The two-consultants analogy. Two consultants write up the same quarter. Both are earnest. The one who understands your business writes a one-pager: *revenue up 4%, driven by Zurich, offset by FX.* The one whose mental model is slightly off writes ten pages of hedging — because every surprise in the data costs her extra sentences. Both describe reality perfectly; only one compresses it. Cross-entropy is a **tax on a wrong model**, paid in pages; written $H(p, q)$ in Theorem 3.1, it is always at least $H(p)$, and the gap is the tax. At 0.53 extra bits per token over a million-token corpus that is roughly **530,000 extra bits of hedging** — a concrete, addable number.

Where the 0.53 **comes from.** Suppose tomorrow's stock move is up 90% of the time, down 10%. The true entropy is small, about 0.47 bits per day. If your model thinks it is 50/50, your average description length becomes 1 bit per day. The 0.53-bit gap is the per-token tax above; the million-token aggregate is the 530,000-bit number.

Perplexity and bits per token. LLM papers report "perplexity 8" or "2.1 bits per token." These are the same number: $\log_2(\text{PPL})$ = bits per token = cross-entropy (Theorem 3.3). Perplexity 8 means the model behaves as if at each word it were choosing uniformly among 8 plausible options. Lower is better.

Why a decision-maker should care: the training objective of every major LLM *is* cross-entropy on text. When a vendor says "our model scored 0.1 lower cross-entropy," they are saying "our model compresses the world 0.1 bits better per word," which over trillions of words is a lot of real modelling skill. Two models with identical cross-entropy are, at the level Shannon cares about, the same model. This gives you a vendor-agnostic yardstick — and a cheap test for benchmark theatre: if a new model does not improve cross-entropy but wins on a specific test, ask whether the test was in the training set.

Sequence models predict *conditionally*. Fix a stochastic process $(X_t)_{t\geq 1}$ on $\mathcal{A}$ and write $X_{<t} = (X_1, \ldots, X_{t-1})$ with $X_{<1} = \emptyset$.

Definition 3.1 (Cross-entropy and conditional cross-entropy)**.** Let p be the true distribution of $(X_1, \ldots, X_T)$ and let q be a model distribution (factoring as $q(x_{1:T}) = \prod_{t=1}^{T} q(x_t \mid x_{<t})$). The **cross-entropy** of q against p over a single symbol with history is

$$H_t(p, q) \;=\; \mathbb{E}_{x_{<t}\sim p}\left[-\sum_{a\in\mathcal{A}} p(a \mid x_{<t})\, \log_2 q(a \mid x_{<t})\right], \tag{7}$$

and the **average cross-entropy per token** is $\overline{H}(p, q) = \frac{1}{T}\sum_{t=1}^{T} H_t(p, q)$.

Theorem 3.2 (Cross-entropy equals expected code length)**.** *Fix T and a model q. Any prefix code that uses $\ell(x_t \mid x_{<t}) = \lceil -\log_2 q(x_t \mid x_{<t})\rceil$ bits to encode x_t given $x_{<t}$ satisfies*

$$\mathbb{E}_{x_{1:T}\sim p}\left[\frac{1}{T}\sum_{t=1}^{T} \ell(x_t \mid x_{<t})\right] \;=\; \overline{H}(p, q) + O(1/T), \tag{8}$$

and arithmetic coding attains (8) *with $O(1/T)$ overhead. Moreover,*

$$\overline{H}(p, q) \;=\; \overline{H}(p) + \overline{D}_{\mathrm{KL}}(p \,\|\, q), \tag{9}$$

where $\overline{H}(p) = \frac{1}{T}\sum_t H(X_t \mid X_{<t})$ *is the true conditional entropy rate and* $\overline{D}_{\mathrm{KL}}(p\|q) \geq 0$ *is the average conditional Kullback–Leibler divergence.*

Proof of (9). For each t and each history $x_{<t}$,

$$-\sum_a p(a\,|\,x_{<t})\log_2 q(a\,|\,x_{<t}) \;=\; -\sum_a p(a\,|\,x_{<t})\log_2 p(a\,|\,x_{<t}) + \sum_a p(a\,|\,x_{<t})\log_2 \frac{p(a\,|\,x_{<t})}{q(a\,|\,x_{<t})}.$$

The first term on the right is $H(X_t \mid X_{<t} = x_{<t})$; the second is $D_{\mathrm{KL}}(p(\cdot \mid x_{<t}) \,\|\, q(\cdot \mid x_{<t}))$. Taking expectation over $x_{<t} \sim p$ and averaging over t yields (9). Non-negativity of the KL term is exactly Theorem 2.3. Statement (8) follows by plugging $\ell = \lceil -\log_2 q \rceil$ into Theorem 2.4 on each conditional distribution and absorbing the at-most-1-bit rounding into the $O(1/T)$ overhead; arithmetic coding eliminates even the integer ceilings up to a constant [Cover and Thomas, 2006, Ch. 5]. □

Corollary 3.3 (Perplexity vs. bits per token)**.** *The* ***perplexity*** *of* q *on* p *is* $\mathrm{PPL}(q) = 2^{\overline{H}(p,q)}$*, so*

$$\log_2 \mathrm{PPL}(q) \;=\; \overline{H}(p,q) \;=\; \textit{(bits per token used by arithmetic coding with model } q\textit{).} \tag{10}$$

Theorem 3.2 is the entire Shannon side of the bridge: *minimising the cross-entropy training objective is literally minimising the expected compressed length.* A next-token predictor with cross-entropy $\overline{H}(p,q)$ bits per symbol corresponds, via arithmetic coding, to a compressor that uses $\overline{H}(p,q)$ bits per symbol. Better predictions ⇔ shorter codes.

A worked numerical example separating the matched and mismatched models on a 3-token sequence is given in Section A.

4 Kolmogorov complexity

Intuition

The complexity of an object is the length of the shortest computer program that prints it — and nothing shorter (see Theorem 4.1).

The recipe-over-the-phone analogy. You are a chef. A colleague in another city must reproduce your dish exactly. You read her the shortest instructions that do the job. "Boil water" takes two words. A soufflé takes a page. A completely random pattern of 10,000 sprinkles has no shorter description than itself: you read out every position. Such an object is *Kolmogorov random*; it has no exploitable structure. Everything else — the digits of π, the works of Shakespeare, your customer database — has a recipe shorter than the raw data.

A tiny worked example. The string `01010101...` (a million long) has a $\sim$ 30-character recipe: `print "01" 500000 times`. A million genuine coin flips has no such shortcut; its recipe is roughly a million characters. Same disk size, wildly different Kolmogorov complexity. Complexity measures *structure*, not *size*.

Invariance and uncomputability. Different programming languages differ in recipe length by at most a constant — the size of a translator (Theorem 4.2). So $K(x)$ is a property of the thing, not your toolkit. But there is no algorithm that, given x, hands you its shortest program (Theorem 4.3); if there were, it would solve the halting problem. $K(x)$ is a perfectly well-defined number you can never compute exactly.

Why a decision-maker should care: K is the theoretical floor every real compressor (zip, JPEG, an LLM) approximates from above. There is a *true* information content in your company's data, vendor- independent. And when we say an LLM "understands" a corpus, one precise meaning is: its weights are a short program reproducing the corpus's statistics — the model approaching K from above.

The compression sniff test

If a vendor claims to compress genuinely random or encrypted data, they are selling snake oil — Shannon closed that door in 1948. A truly random source has entropy equal to its raw bit-length; no code can do better on average (Theorem 2.4). "We compress encrypted traffic by 40%" is either marketing about a non-random side channel (timestamps, packet sizes) or a claim that should not be in the contract.

Shannon's framework is probabilistic: entropy is a property of a distribution, not of an individual string. Kolmogorov's move [Kolmogorov, 1965] was to attach a complexity measure to a *single* object by asking for its shortest description in a fixed universal language.

Fix once and for all a **universal Turing machine** U — a machine that, given as input a description $\langle M, w\rangle$ of any Turing machine M and input w, simulates M on w.

Definition 4.1 (Kolmogorov complexity)**.** The **Kolmogorov complexity** of a finite binary string $x \in \{0,1\}^*$, relative to U, is

$$K_U(x) \;=\; \min\Big\{\, |\pi| \;:\; \pi \in \{0,1\}^*,\; U(\pi) = x \,\Big\}, \tag{11}$$

where $|\pi|$ is the bit-length of the program π, and $U(\pi) = x$ means U on input π halts with output x.

Theorem 4.2 (Invariance theorem)**.** *If U_1 and U_2 are two universal Turing machines, then there exists a constant c_{U_1,U_2}[1], depending on U_1 and U_2 but not on x, such that*

$$\Big|K_{U_1}(x) - K_{U_2}(x)\Big| \;\le\; c_{U_1,U_2} \quad \textit{for every } x \in \{0,1\}^*. \tag{12}$$

Proof sketch. Because U_1 is universal, there is a fixed-length binary program $s_{2\to1}$ that, when prepended to any U_2-program π, causes U_1 to simulate $U_2(\pi)$. Thus every U_2-program π of length $|\pi|$ yields a U_1-program of length $|s_{2\to1}| + |\pi|$ with the same output, giving $K_{U_1}(x) \le K_{U_2}(x) + |s_{2\to1}|$. Symmetrically with $s_{1\to2}$. Taking $c_{U_1,U_2} = \max(|s_{1\to2}|, |s_{2\to1}|)$ gives (12). □

By convention we henceforth write $K(x)$ and treat the universal simulator as the additive constant.

Theorem 4.3 (Uncomputability of K)**.** *There is no total computable function $f : \{0,1\}^* \to \mathbb{N}$ such that $f(x) = K(x)$ for all x.*

[1]The constant c_{U_1,U_2} is finite and does not grow with $|x|$. For pairs of realistic universal machines — two reasonable programming languages interpreting each other — it is typically in the hundreds to low thousands of bits. For data objects much larger than that (a megabyte-scale corpus, an ERP export), it is numerically negligible and "machine-independent up to an additive constant" is a property you can plan around. For objects smaller than the constant, it is not.

Proof (Berry-paradox style). Suppose for contradiction such an f exists. Define the program P_m, parameterised by an integer $m \in \mathbb{N}$, that does the following: enumerate $\{0,1\}^*$ in shortlex order and, for each x, compute $f(x)$ and output the first x such that $f(x) > m$; then halt. Strings of complexity greater than m exist (there are only $2^m - 1$ programs of length less than m, but infinitely many strings), so P_m halts and outputs some string $x_m^\star$ with $K(x_m^\star) > m$.

The program P_m has a fixed body of bit-length B (independent of m) plus a binary encoding of m of at most $\lceil \log_2(m+1) \rceil + O(1)$ bits (e.g. Elias delta). So $|P_m| \leq B + \log_2 m + O(1)$, giving

$$K(x_m^\star) \;\leq\; |P_m| \;\leq\; B + \log_2 m + O(1). \tag{13}$$

Combined with $K(x_m^\star) > m$,

$$m \;<\; B + \log_2 m + O(1), \tag{14}$$

which fails for every sufficiently large m (since $m - \log_2 m \to \infty$), a contradiction. □

The upshot of Theorem 4.3 is that every practical compressor — gzip, PNG, FLAC, and indeed every neural language model — produces an upper bound on $K(x)$, never its exact value.

5 Solomonoff induction

Intuition

To predict the future, average the predictions of every possible theory, giving shorter theories exponentially more weight (Theorems 5.1 and 5.2).

The prediction-market analogy. Imagine a prediction market where every trader is a theory of the world. Simple theories start with a large float; complex theories start with almost nothing — their share price is their *simplicity premium*, set by theory length: a theory of length L bits gets weight 2^{-L}. A theory you can state in 10 words gets a float 1,000 times larger than one needing 20 words. Each new data point is an earnings call. Theories that predicted it cheaply get paid; theories that predicted it badly get margin-called. Over time the market consolidates around the simplest theories that still explain the tape. That consolidation *is* the Solomonoff prediction M. This is Occam's razor, precisely quantified.

A tiny worked example. You see $2, 4, 6, 8$. What's next? The theorist "add 2" has a very short program and shouts "10" at high volume. The theorist "the sequence is $2, 4, 6, 8, 47$, then random" has a longer program and shouts "47" much more quietly. Weighted vote: overwhelmingly "10." If the next number really is 47, all the "add 2" theorists are silenced forever, and the surviving theorists with the shortest explanation of $2, 4, 6, 8, 47$ take over. The machine self-corrects as evidence arrives. A toy two-hypothesis universe is worked out in Theorem 5.3.

Convergence. If the true process is computable, Solomonoff's predictor converges to it: KL error goes to zero (Theorem 5.2). The intuition: the true theory is somewhere in the panel, and every wrong theorist loses microphone share each time she makes a wrong prediction. Eventually the truth dominates.

Why a decision-maker should care: Solomonoff induction is the gold standard of "learning from data." Every ML method is, badly and efficiently, trying to imitate it.

"Bayesian model averaging," "ensemble methods," "regularise for simplicity" — all are engineering approximations of this ideal. It also explains why overfitting is bad in a principled way: an overfit model is a long, ad-hoc theory, and Solomonoff would have weighted it near-zero from the start. The question "which model is best?" has a right answer: whichever predicts held-out data with the fewest bits of surprise.

Kolmogorov complexity selects a single shortest program. Solomonoff's prior [Solomonoff, 1964] performs a *Bayesian average* over all programs, weighted by their lengths. Fix a *prefix-free* universal Turing machine U: the set of halting inputs forms a prefix code, so $\sum_{\pi:U(\pi)\downarrow} 2^{-|\pi|} \leq 1$ by Kraft's inequality.

Definition 5.1 (Universal prior M)**.** For a finite binary string $x \in \{0,1\}^*$, the **universal (Solomonoff) prior** is

$$M(x) \;=\; \sum_{\pi\,:\,U(\pi)=x*} 2^{-|\pi|}, \tag{15}$$

where the sum runs over all programs π such that $U(\pi)$ halts and outputs a string whose prefix is x (denoted $x*$). Equivalently, $M(x)$ is the probability that U fed with a uniformly random infinite bitstream prints x as its first $|x|$ output bits.

The Solomonoff predictive posterior is

$$M(x_{t+1} \mid x_{1:t}) \;=\; \frac{M(x_{1:t}\,x_{t+1})}{M(x_{1:t})}, \tag{16}$$

defined whenever $M(x_{1:t}) > 0$.

Theorem 5.2 (Solomonoff's convergence theorem)**.** *Let μ be any computable probability measure on $\{0,1\}^\infty$. Then there is a constant*

$$c_\mu \;=\; 2^{-K(\mu)+O(1)} \tag{17}$$

such that, for all finite prefixes x,

$$M(x) \;\geq\; c_\mu \cdot \mu(x). \tag{18}$$

Consequently, if $X_{1:\infty} \sim \mu$, the cumulative expected KL divergence between the Solomonoff predictor and μ is bounded uniformly:

$$\sum_{t=1}^{\infty} \mathbb{E}_\mu\Big[D_{\mathrm{KL}}\Big(\mu(\cdot \mid X_{<t}) \,\|\, M(\cdot \mid X_{<t})\Big)\Big] \;\leq\; K(\mu)\ln 2 + O(1). \tag{19}$$

In particular, $D_{\mathrm{KL}}(\mu(\cdot \mid X_{<t}) \| M(\cdot \mid X_{<t})) \to 0$ in μ-expectation as $t \to \infty$.

Proof sketch. Because μ is computable, it has a description as a program π_μ of length $|\pi_\mu| = K(\mu) + O(1)$. Fix any family of programs that, given π_μ as subroutine, simulates a μ-sample to within arbitrarily many bits; summing $2^{-|\pi|}$ over that family and restricting to those whose output begins with x shows $M(x) \geq 2^{-K(\mu)-O(1)}\,\mu(x)$, which is (18) with c_μ as in (17). Taking logarithms and applying a telescoping chain-rule argument to the KL divergence then yields (19). The correct program π_μ has prior mass exactly $2^{-K(\mu)}$; the finite KL budget is the information-theoretic price of identifying it. For the full proof see Li and Vitányi [2019, Ch. 5]. □

Example 5.3 (Toy Solomonoff update on a two-program universe)**.** Restrict the universe of programs to two deterministic generators: π_0 : always output 0 (bit-length $|\pi_0| = 2$) and π_1 : output the cycle 01010101 . . . (bit-length $|\pi_1| = 3$). Under the Solomonoff-style weighting $w(\pi) = 2^{-|\pi|}$, the prior over the two hypotheses is

$$P(\pi_0) = \frac{2^{-2}}{2^{-2} + 2^{-3}} = \frac{2}{3}, \qquad P(\pi_1) = \frac{2^{-3}}{2^{-2} + 2^{-3}} = \frac{1}{3}.$$

Observe the prefix $x_{1:4} = 0101$. Likelihoods: $\Pr(0101 \mid \pi_0) = 0$ (the all-zeros generator cannot produce a 1); $\Pr(0101 \mid \pi_1) = 1$. Bayes' rule gives $P(\pi_0 \mid 0101) = 0$, $P(\pi_1 \mid 0101) = 1$. So the Solomonoff predictor assigns $M(x_5 = 0 \mid 0101) = 1$, $M(x_5 = 1 \mid 0101) = 0$. Two features: the shorter program π_0 was *not* favoured after data — simplicity is a prior preference overridden by evidence — and a single contradicting bit collapses an exponentially-weighted prior to a point mass, which is the mechanism by which Solomonoff induction converges so fast on deterministic sources.

6 AIXI: intelligence as optimal compression-driven prediction

Intuition

The theoretically optimal agent is the one that, before each action, asks the Solomonoff panel what is likely to happen, and picks the action whose expected future reward is highest (Theorems 6.1 and 6.2).

The regional-manager analogy. Imagine a regional manager who, before every staffing decision, consulted every possible theory of her stores — every demand model, every weather-effect hypothesis, every crank manifesto about Mercury retrograde — weighted by how simple each theory is. For each candidate decision she asks: "If I do this, what happens over the next hundred Tuesdays, and how much will I enjoy it?" She takes the weighted average and places the staffing bet that maximises expected sales across all of them. That manager would be provably optimal in a precise technical sense. She would also be impossible: consulting every possible theory takes infinite time. This is *AIXI*. It is a **north star for what a learner is approximating, not a machine anyone can build**. Real systems are food trucks reading the Michelin menu; AIXI is the chef with an infinite pantry and no kitchen.

A tiny worked example. A two-armed bandit: Arm A paid out once in three pulls; Arm B paid out zero times in one pull. The simplest theories favour A; slightly more complex theories say B was just unlucky. Weight theories by simplicity, compute expected reward from each arm over the next 100 pulls, pick the winner. You will usually pull A but occasionally explore B — not because someone programmed exploration, but because the simplicity-weighted average *naturally* keeps B alive. AIXI does this for every decision, everywhere.

Why Hutter calls it "universal intelligence." AIXI does not need to know the game, the physics, or the rules. Drop it into any computable environment and it will eventually learn the environment and play optimally. Its only prior is Occam's razor.

The catch. AIXI is uncomputable; it requires running every possible program. Bounded approximations exist but are toys at present. AIXI is to intelligence what the ideal gas law is to a real gas — a north star, not a product.

Why a decision-maker should care: AIXI is a crisp, non-hand-wavy definition of "what would a perfectly general AI do?" When someone claims their system is "a step toward AGI," ask: is it a step toward better approximating AIXI? Is it building better world-models and planning against them? If yes, the claim is in the right ballpark. If it is pattern-matching in a narrow domain with no planning layer, the claim is marketing. And no real system *is* AIXI, so any product advertised as "universal intelligence" is abusing the term.

Hutter's AIXI framework [Hutter, 2005] extends Solomonoff induction from passive sequence prediction to sequential decision-making. The agent interacts with an environment over discrete time steps. At step t the agent chooses an action $a_t \in \mathcal{A}_{\text{act}}$, and the environment returns a percept $e_t = (o_t, r_t)$ consisting of an observation $o_t \in \mathcal{O}$ and a reward $r_t \in [0, r_{\max}] \subset \mathbb{R}$. Write the interaction history as $h_{<t} = (a_1, e_1, a_2, e_2, \ldots, a_{t-1}, e_{t-1})$.

An **environment** is a conditional probability measure $\nu(e_{1:m} \mid a_{1:m})$ over percept sequences given action sequences. The **universal mixture environment** is

$$\xi(e_{1:m} \mid a_{1:m}) \;=\; \sum_{\nu \in \mathcal{M}} 2^{-K(\nu)}\, \nu(e_{1:m} \mid a_{1:m}), \tag{20}$$

where $\mathcal{M}$ is the class of all lower-semicomputable ("chronological") environments and $K(\nu)$ is the Kolmogorov complexity of a prefix program for ν.

Definition 6.1 (AIXI)**.** Fix a finite planning horizon m and a discount. The **AIXI action** at history $h_{<t}$ is

$$a_t^{\text{AIXI}} \;=\; \arg\max_{a_t} \sum_{e_t} \max_{a_{t+1}} \sum_{e_{t+1}} \cdots \max_{a_m} \sum_{e_m} \Big(\sum_{k=t}^{m} r_k \Big)\, \xi\Big(e_{t:m} \,\Big|\, h_{<t}, a_{t:m}\Big). \tag{21}$$

In words: AIXI plans by expectimax over all possible futures, weighted by the Solomonoff-type universal mixture (20). The $2^{-K(\nu)}$ weighting is what ties the definition of intelligence back to compression: hypothesised environments with short descriptions receive exponentially more weight, and an AIXI-optimal agent simultaneously (i) compresses its environment optimally and (ii) acts to maximise reward under that compressed model.

Theorem 6.2 (Universal optimality of AIXI, informal)**.** *Within the class of lower-semicomputable environments, AIXI is* Pareto optimal*: no agent π' achieves strictly greater expected total reward than AIXI in every environment $\nu \in \mathcal{M}$. Moreover, for any computable ν the AIXI predictor converges to the true environment's predictions in the sense of* (19) *with M replaced by ξ.*

For the precise statement and proof see Hutter [2005, Ch. 5]. The significance of Theorem 6.2 is that "intelligence" has, within this framework, been reduced to a single ingredient (the universal prior (20)) plus bookkeeping (the expectimax expansion). Remove the Solomonoff prior and the agent collapses to uninformed search; keep it, and the agent becomes universally optimal.

7 What this buys us for LLMs

Intuition

An LLM trained to predict the next word is, mechanically and provably, a compressor of its training data — which, per the chain above, puts it somewhere on the intelligence spectrum (Equation (23) and Theorems 3.2 and 5.2).

Residuals and arithmetic coding, in plain English. A *residual* is the gap between what the model predicted and what actually came next. If the model gave next-token probability 0.9 to "the" and "the" showed up, the residual is tiny — a fraction of a bit. If the model gave 0.01 and "the" still showed up, the residual is large — about 6.6 bits of surprise. *Arithmetic coding* is just a near-optimal way to write all those surprise numbers down end to end. The compressed file is therefore the model's weights (fixed cost) plus the bill for every surprise (variable cost). "KL divergence", when it appears below, is the same tax formalised: the average extra bits per token you pay for using the wrong distribution.

The Hutter Prize as a concrete hinge. Marcus Hutter put up real prize money for whoever can compress 1 GB of Wikipedia the most. The current leaderboard is dominated by neural compressors. Training an LLM on Wikipedia to minimise cross-entropy *is* compressing Wikipedia; the model weights plus arithmetic-coded residuals are the compressed file. Better prediction = smaller file = (by Theorem 3.2) better distributional model = (by Theorem 5.2) closer to the true generating process = (per Hutter) more intelligence.

Scaling laws. Plot model size against cross-entropy loss on held-out text: you get a strikingly clean curve. Double the data, double the parameters, loss drops predictably. The curve has been stable across four orders of magnitude [Kaplan et al., 2020]. Cross-entropy — our bits-per-token yardstick — is a smooth function of investment. You can buy intelligence by the bit. Not efficiently, not forever, and with diminishing returns — but predictably.

A tiny worked example. GPT-2 hit roughly 1.2 bits per English character on a standard benchmark; GPT-4-class models are closer to 0.7. That looks like a modest drop. But English text is about 1.3 bits of true entropy per character (Shannon's 1950s estimate). So GPT-2 was $\sim 92\%$ of the way to the floor; GPT-4 is $\sim 98\%$. Small in bits, enormous in dollars — the last bit contains the reasoning, the rare facts, the nuanced world-models.

Emergent capabilities, cautious framing. If the training objective is a compression proxy and compression is provably linked to a universal notion of intelligence, then LLMs doing arithmetic, translation, and code — capabilities nobody explicitly trained for — is not a bug. It is the theory predicting itself. To compress text well, you must model the world the text is about.

Why a decision-maker should care, three upshots. First, you now have a real yardstick — cross-entropy on your own domain data — to compare vendors. Second, you understand why "just predict the next word" produces general-looking behaviour; it is not marketing, the mathematics agrees. Third, you can reason about the ceiling: as models approach the true entropy of text, progress on pure language tasks must slow, and the frontier moves to multimodal data, embodied interaction, and reasoning architectures. Plan capital allocation accordingly.

Concretely, a large language model parameterises a conditional distribution $q_\theta(x_t \mid x_{<t})$

and is trained by minimising the negative log-likelihood

$$\mathcal{L}(\theta) \;=\; -\frac{1}{T}\sum_{t=1}^{T}\log_2 q_\theta(x_t \mid x_{<t}). \tag{22}$$

By Theorem 3.2, $\mathcal{L}(\theta)$ is the expected arithmetic-coding length per token: minimising cross-entropy *is* minimising compressed size. Equivalently, by Theorem 3.3, lowering perplexity lowers bits-per-token by the same amount.

Kaplan et al. [2020] showed that $\mathcal{L}(\theta)$ follows clean power laws in model size N, dataset size D, and compute C, of the form $\mathcal{L} \approx (N_c/N)^{\alpha_N}$. Viewed through (22), each such power-law decrement is a *bit-per-token compression improvement*. The operational content of Theorem 5.2 in this setting is that, to the extent that the data-generating process is computable, any predictor whose bits-per-token approaches the conditional entropy rate $\overline{H}(p)$ is implicitly approximating the μ-posterior of the universal prior M.

Delétang et al. [2024] operationalised this equivalence by wiring a large language model into an arithmetic coder and compressing raw bytes of image and audio data. They reported compression ratios lower than domain-specific specialist compressors (e.g. PNG, FLAC) on out-of-domain modalities, despite the model having been trained solely on text. From the vantage point of Theorems 3.2 and 5.2, the experiment is a consistency check: a predictor that approximates a universal prior should, by (18), compress any computable source to within an additive $K(\mu)$ of its true entropy, and text exposure is enough to learn a useful approximation of that prior.

The **Hutter Prize** [Hutter, 2005] formalises the economic version of the claim: a cash prize for improving the lossless compression ratio on a fixed text corpus. Within the equivalence framework, every winning submission is, by definition, a model whose cross-entropy on the corpus is lower — which, by Theorem 3.3, is a more accurate next-symbol predictor.

Collecting the theorems of Sections 3 to 6 we obtain the *working equivalence*:

$$\underbrace{\text{low cross-entropy}}_{\text{good predictor}} \iff \underbrace{\text{short arithmetic code}}_{\text{good compressor}} \iff \underbrace{\text{approx. of universal prior}}_{\substack{\text{Solomonoff/AIXI}\\ \text{sense of intelligence}}} \tag{23}$$

The double arrows are theorems within Shannon's framework (Theorem 3.2) and asymptotic theorems modulo computability (Theorem 5.2).

8 Caveats and what the equivalence does not say

Intuition

The equivalence is real but narrow: it tells you that good prediction entails good *modelling*, not that good prediction entails understanding, agency, or safety. It promises statistical competence. It does not promise causal reasoning, counterfactual simulation, or moral judgement. Those may emerge — the theory leaves the door open — but they are not guaranteed by the objective.

Uncomputability is not cosmetic. $K(x)$ and AIXI are unreachable (Theorem 4.3). Every real compressor and every real agent is a finite approximation. Saying an LLM is "approximating AIXI" is like a food-truck claiming to approximate a Michelin three-star kitchen by reading the menu — technically on the same axis, in practice

not the relevant comparison.

The only non-tautological falsifier. A reasonable operator will ask: if AIXI is uncomputable and every deployed system is "a finite approximation", what test other than cross-entropy itself decides that system A approximates the ideal better than system B? Honest answer: held-out cross-entropy on distributions the training set did not see, plus downstream task transfer. There is no deeper falsifier in the current theory. If that makes the AIXI framing feel tautological to an operator, the operator is reading the state of the field correctly.

Compression $\neq$ understanding. A junior analyst who has memorised every deal memo in the vault — say, 4,000 deals going back a decade — has compressed a lot of data. Ask her to price a deal that is not in the vault and you find out whether she compressed the memos or the market. A lossless compressor has captured statistics; that is *some* kind of understanding, but not necessarily the kind a manager means when she asks "does the model understand our business?"

Embodiment. A text-only system has only ever been surprised by text. It has no ground-truth feedback from physical reality. Solomonoff's framework accepts any computable environment, but LLMs have the text channel and lack the body-in-a-world channel. Agentic, tool-using, multimodal systems are early attempts to close that gap.

Benchmarks are not the floor. A model with low cross-entropy on a benchmark may just have memorised the benchmark. Data contamination is rampant. The equivalence only holds when the test distribution is genuinely held out.

Why a decision-maker should care: use the equivalence as a *lower bound*, not an upper bound. When a vendor says "lowest perplexity," accept that as real evidence of genuine modelling skill — and then ask the second question: skill at what, tested on what, does it translate to your domain? The equivalence tells you the knob exists. It does not tell you that turning the knob solves your problem. Confusing "next-token prediction" with "just autocomplete" misses a deep result; confusing "next-token prediction" with "general intelligence" overclaims a narrow one. Both errors are expensive. Staying in the middle — taking the equivalence seriously but not theologically — is the managerial win.

Several limitations of the chain (23) deserve formal acknowledgement.

(C1) Uncomputability. By Theorem 4.3, $K(x)$ is not computable; by a related argument so is $M(x)$. The entire right-hand side of (23) therefore names a mathematical ideal, not an algorithm. Every real compressor produces an upper bound $\tilde{K}(x) \geq K(x)$, and equality is in principle unattainable. "Approximates the universal prior" must be taken as an asymptotic statement about cross-entropy, not as a literal claim that the model is computing M.

(C2) The additive constant. Theorem 4.2 only guarantees machine-independence up to an additive constant c_{U_1,U_2}. For short strings this constant can dominate $K(x)$ itself; Kolmogorov complexity is only meaningful asymptotically. Analogously, the $K(\mu)$ term in (19) is a finite one-time information cost: it does not shrink with data.

(C3) Practical compressors vs. K. A compressor C is *universal* in Shannon's sense if its code length $L_C(x_{1:T})/T$ converges to the entropy rate of any stationary ergodic source.

It is *not* in general universal in Solomonoff's sense: it need not dominate all computable measures. The gap between Lempel–Ziv (Shannon-universal) and Solomonoff (universal over all computable measures) is a full logical level of universality, and neural compressors live somewhere in between.

(C4) Necessary vs. sufficient. Theorems 5.2 and 6.2 establish that optimal prediction and agency *entail* optimal compression. They do *not* establish that every improvement in compression corresponds to an improvement in every intuitive sense of "understanding." In particular, the framework is silent on phenomenal consciousness, grounding, and embodiment, and it takes the reward signal and the action–percept loop as primitives rather than explaining their origin.

(C5) Compression $\neq$ understanding (as internal structure). A compressor is characterised by its length function, not by its internal representations. Two compressors with identical code lengths may use entirely different internal mechanisms, only one of which we would intuitively call "understanding." Equation (23) reduces "intelligence" to an extensional (input–output) property; any argument that understanding is intensional (about how the function is computed) is not touched by the chain.

Summary. The equivalence (23) is a genuine theorem modulo Theorems 2.1 to 6.1 and the computability-based caveats (C1)–(C3). What it licences is a precise sense in which lowering cross-entropy on a sufficiently rich corpus moves a predictor along a single axis shared with compression and with Hutter-style universal agency. What it does *not* licence is the stronger claim that this axis exhausts what we mean by understanding.

Further reading

Information theory, foundations.
Shannon [1948] is the founding paper and still the clearest exposition of the source-coding setup. Cover and Thomas [2006] is the modern textbook reference; chapters 2–5 cover everything in Sections 2 and 3.

Kolmogorov complexity and Solomonoff induction.
Kolmogorov [1965] introduces the three approaches to defining information; Solomonoff [1964] introduces the universal prior. Li and Vitányi [2019] is the definitive modern textbook, with full proofs of Theorems 4.2, 4.3 and 5.2.

Universal agency.
Hutter [2005] is the original AIXI monograph, including proofs of Theorem 6.2 and related optimality results.

LLMs as compressors, empirical.
Kaplan et al. [2020] gives the scaling-law curves discussed in Section 7. Delétang et al. [2024] demonstrates experimentally that a text-trained large language model, used as the model inside an arithmetic coder, compresses out-of-domain byte streams (images, audio) competitively.

A Worked cross-entropy calculation

Example A.1 (Cross-entropy on a 3-token sequence; nats vs. bits)**.** Let $\mathcal{A} = \{A, B, C\}$ and suppose the true conditional distributions at positions $t = 1, 2, 3$ have already been conditioned on the observed prefix, giving the sequence of true posterior masses on the realised tokens $p_1 = 0.8,\ p_2 = 0.5,\ p_3 = 0.1$. Consider two models. Model $q^{(1)}$ assigns $q_1 = 0.8, q_2 = 0.5, q_3 = 0.1$ (matches truth on the realised tokens); model $q^{(2)}$ assigns $q_1 = 0.1, q_2 = 0.1, q_3 = 0.8$ (systematically wrong).

For model $q^{(1)}$, the per-token code lengths in bits are $-\log_2 0.8 \approx 0.3219$, $-\log_2 0.5 = 1$, $-\log_2 0.1 \approx 3.3219$, summing to approximately 4.644 bits. In nats, multiply by $\ln 2 \approx 0.6931$: $4.644 \cdot \ln 2 \approx 3.219$ nats, matching $-\ln(0.8 \cdot 0.5 \cdot 0.1) = -\ln 0.04 \approx 3.219$ directly.

For model $q^{(2)}$, the lengths are $-\log_2 0.1 \approx 3.3219$, $-\log_2 0.1 \approx 3.3219$, $-\log_2 0.8 \approx 0.3219$, totalling ≈ 6.966 bits (≈ 4.828 nats, consistent with $-\ln(0.008) \approx 4.828$). The wrong model spends 2.32 extra bits, the Kullback–Leibler excess predicted by (9). Conversion rule: 1 nat $= \log_2 e$ bits ≈ 1.4427 bits and conversely 1 bit $= \ln 2$ nats.

References

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2 edition, 2006.

Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.10668.

Marcus Hutter. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2005. doi: 10.1007/b138233.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Andrey N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

Ming Li and Paul M. B. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Springer, Cham, 4 edition, 2019. doi: 10.1007/978-3-030-11298-1.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

Ray J. Solomonoff. A formal theory of inductive inference. Parts I and II. *Information and Control*, 7(1–2):1–22, 224–254, 1964. doi: 10.1016/S0019-9958(64)90223-2.